

Secuestro de PATH

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Cuando un programa privilegiado invoca otro comando por su nombre (por ejemplo 'cat' en vez de '/bin/cat'), el sistema lo busca por el PATH. Si puedes modificar el PATH o escribir en un directorio que aparezca antes, colocas ahí un binario malicioso con ese nombre y el proceso root ejecuta el tuyo. Otro caso del privesc-modelo.

El programador uso una ruta relativa por comodidad, confiando en que el PATH apunta a los binarios legítimos. Pero si controlas el PATH (o hay un directorio escribible en el), rompes esa confianza: tu binario se ejecuta con los privilegios del proceso.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando un SUID (suid) o un script de cron-abuse que corre como root llama a comandos sin ruta absoluta. strings sobre el binario o leer el script lo revela.

Cómo identificarla

- strings de un SUID muestra un comando llamado sin ruta (system("service ...")).
- Un script root usa comandos sin path absoluto.
- Tienes un directorio escribible que puedes anteponer al PATH.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

strings sobre el SUID y busca llamadas a comandos sin '/' delante.

Evidencia esperada: Un nombre de comando sin ruta absoluta en las strings; tras crear tu binario falso y anteponerlo al PATH, ejecución como el proceso privilegiado.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
strings /ruta/al/suid
```

Objetivo y contexto: Revela cadenas del binario, incluidos comandos que llama. Un comando sin '/' es candidato a PATH hijacking; con ruta absoluta, no.

Resultado esperado: Nombres de comandos invocados. Si ves 'service', 'cat', 'ps' sin ruta, ese es tu punto: crea un binario con ese nombre en un dir que controles.

Capa 2: prueba documentada

```
cd /tmp && echo '/bin/bash' > cat && chmod +x cat && export PATH=/tmp:$PATH
```

Objetivo y contexto: Crea un 'cat' falso que lanza bash, y lo pone primero en el PATH. Cuando el proceso root llame a 'cat', ejecutara el tuyo con privilegios de root.

Resultado esperado: Al ejecutar despues el binario/script privilegiado, obtienes shell root. Restaura el PATH luego para no romper tu propia sesion.

Anatomía de los payloads

La primera prueba es `strings /ruta/al/suid`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un nombre de comando sin ruta absoluta en las strings; tras crear tu binario falso y anteponerlo al PATH, ejecucion como el proceso privilegiado.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Detecta el comando invocado sin ruta absoluta (strings o leyendo el script).
- Crea un binario/script con ese nombre que lance una shell (o ponga SUID a bash).
- Antepone tu directorio al PATH (export PATH=/tmp:\$PATH) y dale permiso de ejecucion.
- Ejecuta el binario/script privilegiado; correra tu version. Confirma uid=0.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El binario/script usa siempre rutas absolutas (/bin/cat, /usr/bin/service).

- El proceso privilegiado fija su propio PATH explícitamente antes de ejecutar (no hereda el tuyo).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si un script o SUID que corre como root llama a un binario sin ruta absoluta, colocas tu propio binario antes en el PATH. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (path-hijacking)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.